



Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»

ФАКУЛЬТЕТ «Программной инженерии и компьютерной техники»

ОБРАЗОВАТЕЛЬНАЯ ПРОГРАММА «Проектирование и разработка систем больших данных»

НАПРАВЛЕНИЕ ПОДГОТОВКИ (СПЕЦИАЛЬНОСТЬ) «09.04.04 Программная инженерия»

ОТЧЕТ

по лабораторной работе №5

по курсу «Структуры и алгоритмы в базах данных и распределенных
системах»

на тему: «Реализация обратного индекса»

Обучающийся: Постнов С. А., Р4135
(Ф.И.О, № группы)

Преподаватель Платонов А. В., доцент
(Ф.И.О, должность)

Преподаватель Портнов П. В., ассистент
(Ф.И.О, должность)

2026 г.

СОДЕРЖАНИЕ

1	Теоретическая часть	3
1.1	Обратный индекс	3
1.2	Операции над posting lists	3
1.3	Skip lists	3
1.4	Дельта-кодирование	4
1.5	Ранжирование BM25	4
2	Практическая часть	5
2.1	Архитектура реализации	5
2.2	Формат хранения индекса	6
2.3	Поиск	6
3	Исследовательская часть	7
3.1	Аппаратные характеристики	7
3.2	Методика замеров	7
3.3	Результаты построения индекса	8
3.4	Результаты wairmir	9
3.5	Результаты поиска в индексе	10
3.6	Размер индекса	12
3.7	Профилирование	13
4	Вывод	17

1 Теоретическая часть

1.1 Обратный индекс

Обратный индекс (inverted index) — структура данных, отображающая термы в список документов (posting list), в которых они встречаются. Для каждого терма хранится упорядоченный по DocID список постингов, содержащих идентификатор документа и позиции вхождения терма. Обратный индекс является основной структурой данных в поисковых системах, обеспечивая быстрый поиск документов по ключевым словам.

Формат постинга: $(DocID, [pos_1, pos_2, \dots, pos_k])$, где $DocID$ — идентификатор документа, pos_i — позиции вхождения терма в документе. Позиции необходимы для реализации операторов (ADJ, NEAR).

1.2 Операции над posting lists

Основные операции над posting lists, используемые в поисковых запросах:

- *Intersect* $(A \cap B)$ — пересечение документов (оператор AND);
- *Union* $(A \cup B)$ — объединение документов (оператор OR);
- *Difference* $(A \setminus B)$ — разность документов (оператор NOT);
- *Adjacent* — документы, где термы стоят подряд (оператор ADJ);
- *Near* — документы, где термы стоят на расстоянии $\leq k$ (оператор NEAR/ k).

1.3 Skip lists

Skip list — вероятностная структура данных поверх упорядоченного списка, позволяющая выполнять поиск за $O(\log(n))$ вместо $O(n)$. В данной реализации skip list строится детерминированно: на уровне i размещаются элементы с шагом 2^{i+1} , начиная с $\lceil \sqrt{n} \rceil$. Итератор PostingIterator поддерживает курсоры на каждом уровне и метод AdvanceTo(docID), который перемещает итератор к первому элементу $\geq docID$, используя skip-указатели для быстрого продвижения.

1.4 Дельта-кодирование

Для сжатия posting lists применяется дельта-кодирование (delta encoding), то есть вместо абсолютных значений DocID и позиций хранятся разности между соседними элементами. Поскольку DocID упорядочены, а позиции внутри документа также упорядочены, разности малы и эффективно кодируются переменным числом байт (`binary.PutUvarint`).

1.5 Ранжирование BM25

BM25 — функция ранжирования, вычисляющая релевантность документа запросу:

$$\text{score}(D, Q) = \sum_{t \in Q} \text{IDF}(t) \cdot \frac{f(t, D) \cdot (k_1 + 1)}{f(t, D) + k_1 \cdot (1 - b + b \cdot |D| / \text{avgdl})}$$

, где $f(t, D)$ — частота термина t в документе D , $|D|$ — длина документа, avgdl — средняя длина документа, $k_1 = 1.5$ и $b = 0.75$ — параметры.

$$\text{IDF}(t) = \ln(1 + (N - n(t) + 0.5) / (n(t) + 0.5))$$

, где N — число документов, $n(t)$ — число документов, содержащих терм t .

2 Практическая часть

2.1 Архитектура реализации

Проект реализован на Go и состоит из следующих пакетов:

1) `internal/index`:

- `IndexBuilder` — построение индекса, добавление документов, токенизация, построение posting lists, дельта-кодирование;
- `InvertedIndex` — загрузка индекса через `mmap`, декодирование posting list;
- `PostingList` — постинг-лист со skip list и итератором `PostingIterator`;
- операции `Intersect`, `Union`, `Difference`, `Adjacent`, `Near`.

2) `internal/query`:

- `Parser` — рекурсивный парсинг запроса с операторами `AND`, `OR`, `NOT`, `ADJ`, `NEAR/k` и скобками;
- `Evaluator` — обход AST и вызов соответствующих операций над posting lists.

3) `internal/ranking` — ранжирование BM25;

4) `internal/compression` — дельта-кодирование/декодирование posting lists;

5) `internal/storage` — двоичный формат индекса (header + term entries + postings data) и хранилище документов (`DocStore`);

6) `internal/engine` — поисковый движок, объединяющий все необходимые компоненты;

7) `cmd/tui` — TUI-интерфейс на `bubbletea`.

2.2 Формат хранения индекса

Индекс хранится в двух файлах:

- `*.idx` — сам индекс с заголовком (40 байт: magic, версия, число термов, число документов, число токенов, смещения), отсортированными term entries и сжатыми posting lists;
- `*.docs` — хранилище документов.

Обратный индекс загружается через `mmap` и читает только заголовок и term entries, а posting lists декодируются лениво при поиске. Страницы `mmap` подгружаются операционной системой при первом обращении.

2.3 Поиск

Обработка запроса состоит из трёх этапов:

- 1) *Парсинг*: строка запроса разбирается в AST;
- 2) *Вычисление*: обход AST, для каждого терма загружается posting list, операции выполняются в соответствии с приоритетом слева направо;
- 3) *Ранжирование*: для документов из результирующего posting list вычисляется BM25 score.

3 Исследовательская часть

3.1 Аппаратные характеристики

Все замеры проводились на ноутбуке со следующей конфигурацией:

- операционная система: macOS (arm64);
- процессор: Apple M3 Pro, 11 ядер;
- оперативная память: 18 ГБ;
- версия Go: go1.25.

Во время замеров не выполнялись ресурсоёмкие фоновые задачи, а ноутбук был подключён к сети электропитания.

3.2 Методика замеров

Для замеров использованы встроенные benchmark-тесты Go. Измерялись три операции:

- **build** — построение индекса;
- **warmup** — прогрев индекса;
- **search** — поиск по запросам различных типов.

Для построения индекса использованы документы размером 500, 700, 1000, 2000, 3000, 4000, для поиска и warmup — документы размером 10000, 25000, 50000, 75000, 100000, 125000, 150000, 175000. Разделение размеров обусловлено тем, что построение индекса при больших N не успевает выполнить более одной итерации за таймаут.

Для каждого размера фиксировались:

- среднее время на операцию (нс/оп);
- 95% доверительный интервал (CI95);
- размер индекса в байтах (для build).

Поиск измеряется для следующих типов запросов: Term, And, Or, Not, Adj, Near, а также составных: Complex_AndOr, Complex_AdjAnd, Complex_AndNot.

3.3 Результаты построения индекса

Таблица 3.1 содержит результаты замеров времени построения индекса и его размера.

Таблица 3.1 – Время построения индекса и размер

N	время (мс)	размер (МБ)
500	296 ± 57	17
700	410 ± 73	22
1000	495 ± 103	31
2000	840 ± 53	57
3000	1197 ± 47	85
4000	1615 ± 77	112

График времени построения индекса представлен на рисунке 3.1.

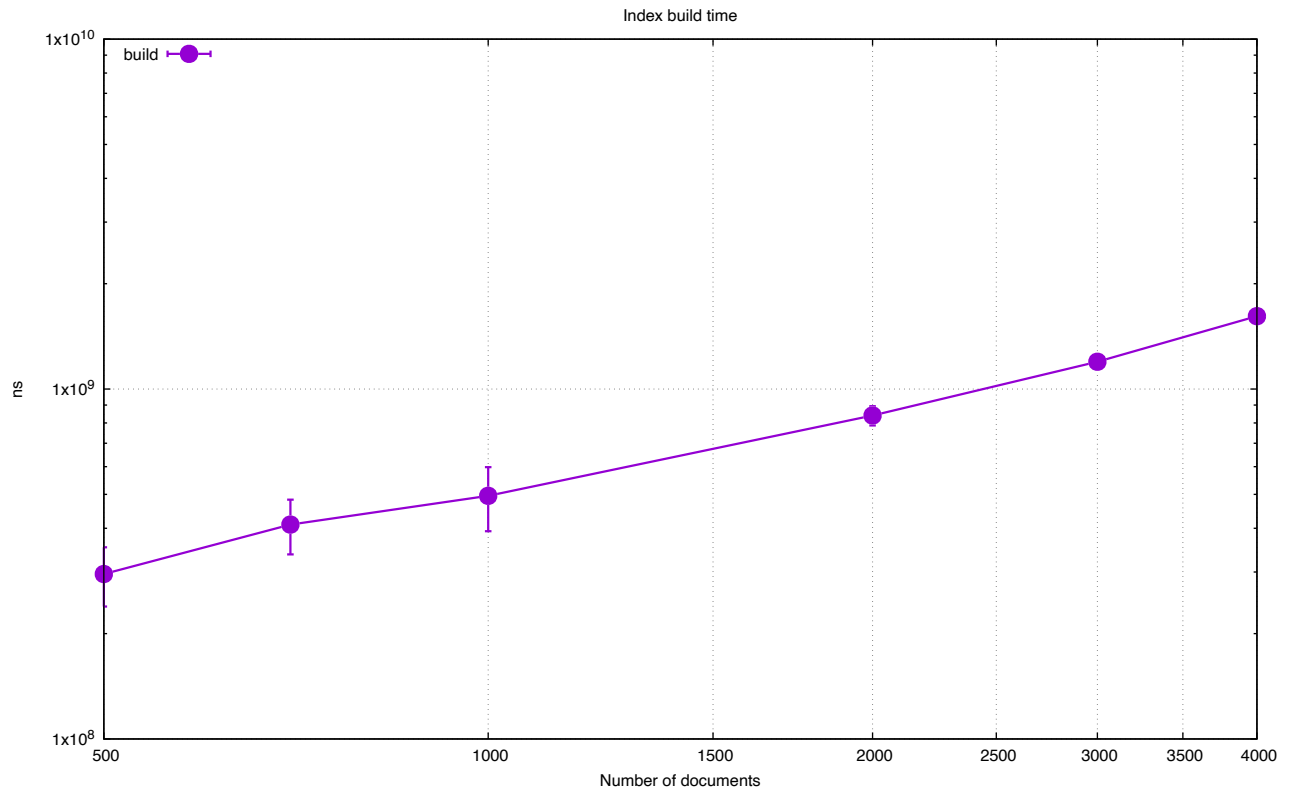


Рисунок 3.1 – Время построения индекса в зависимости от числа документов

Время построения растёт линейно. При увеличении N в 8 раз (от 500 до 4000) время увеличивается примерно в 6 раз (от 296 до 1615 мс).

3.4 Результаты warmup

Таблица 3.2 содержит результаты замеров прогрева индекса.

Таблица 3.2 – Время warmup (мс)

N	время (мс)
10000	179 ± 63
25000	449 ± 37
50000	683 ± 64
75000	735 ± 87
100000	963 ± 31
125000	1060 ± 55
150000	1166 ± 34
175000	1427 ± 63

График времени warmup представлен на рисунке 3.2.

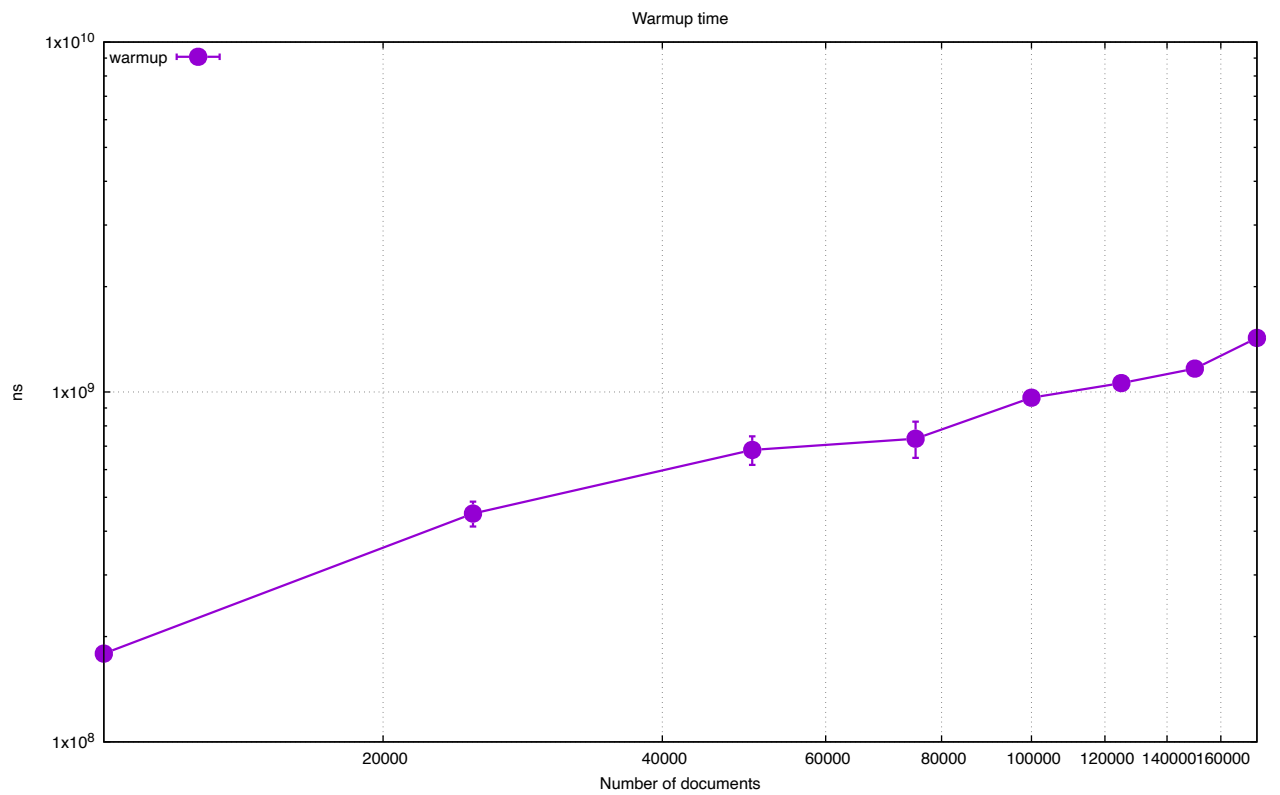


Рисунок 3.2 – Время первого запроса после загрузки индекса (warmup)

Время warmup значительно превышает время повторного поиска. Это объясняется тем, что при первом обращении к posting lists операционная систе-

ма подгружает страницы из диска в память. С ростом N время `warmup` растёт, что объясняется увеличением числа `mmap`-страниц, к которым обращается поисковый запрос.

3.5 Результаты поиска в индексе

Таблица 3.3 содержит результаты замеров времени базовых операций поиска.

Таблица 3.3 – Время базовых операций поиска (мс)

N	Term	And	Or	Not	Adj	Near
10000	13 ± 1	9 ± 0.1	10 ± 0.1	10 ± 0.1	19 ± 0.2	25 ± 0.2
25000	23 ± 1	18 ± 1	23 ± 1	22 ± 0.2	40 ± 0.4	51 ± 1
50000	31 ± 1	24 ± 1	32 ± 0.4	32 ± 0.4	54 ± 1	68 ± 1
75000	53 ± 5	34 ± 2	43 ± 1	42 ± 0.5	68 ± 1	85 ± 1
100000	53 ± 2	35 ± 1	52 ± 1	52 ± 1	82 ± 1	102 ± 1
125000	78 ± 7	44 ± 2	63 ± 1	62 ± 1	96 ± 1	119 ± 2
150000	74 ± 3	50 ± 3	73 ± 1	72 ± 1	110 ± 2	136 ± 2
175000	109 ± 10	57 ± 3	83 ± 1	82 ± 1	125 ± 2	155 ± 2

Таблица 3.4 содержит результаты замеров времени составных операций поиска.

Таблица 3.4 – Время составных операций поиска (мс)

N	OrAnd	AdjAnd	AndNot
10000	25 ± 0.4	21 ± 0.2	20 ± 0.2
25000	51 ± 1	44 ± 1	42 ± 1
50000	67 ± 1	58 ± 1	57 ± 1
75000	82 ± 2	71 ± 1	73 ± 1
100000	98 ± 2	86 ± 1	88 ± 2
125000	113 ± 2	99 ± 1	104 ± 2
150000	129 ± 3	113 ± 2	120 ± 2
175000	146 ± 4	128 ± 2	137 ± 2

Графики времени поиска представлены на рисунках 3.3 и 3.4.

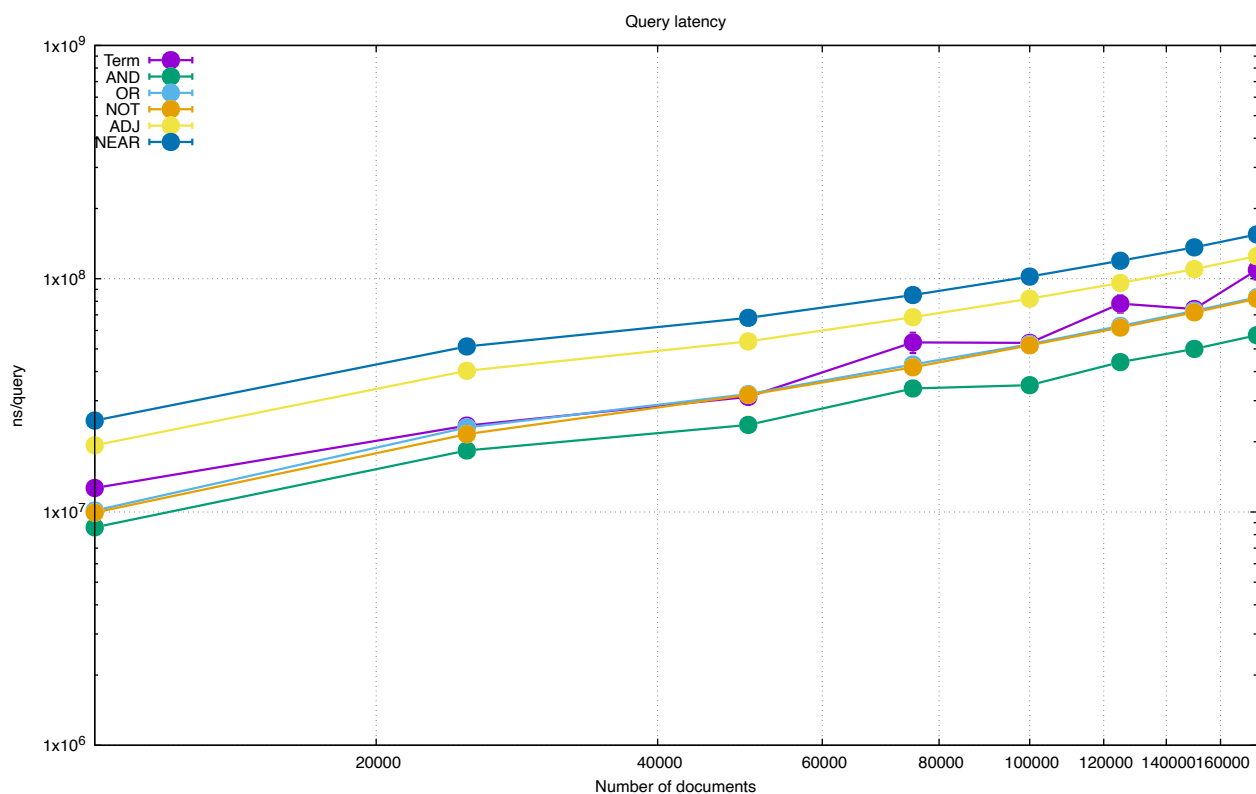


Рисунок 3.3 – Время базовых операций поиска

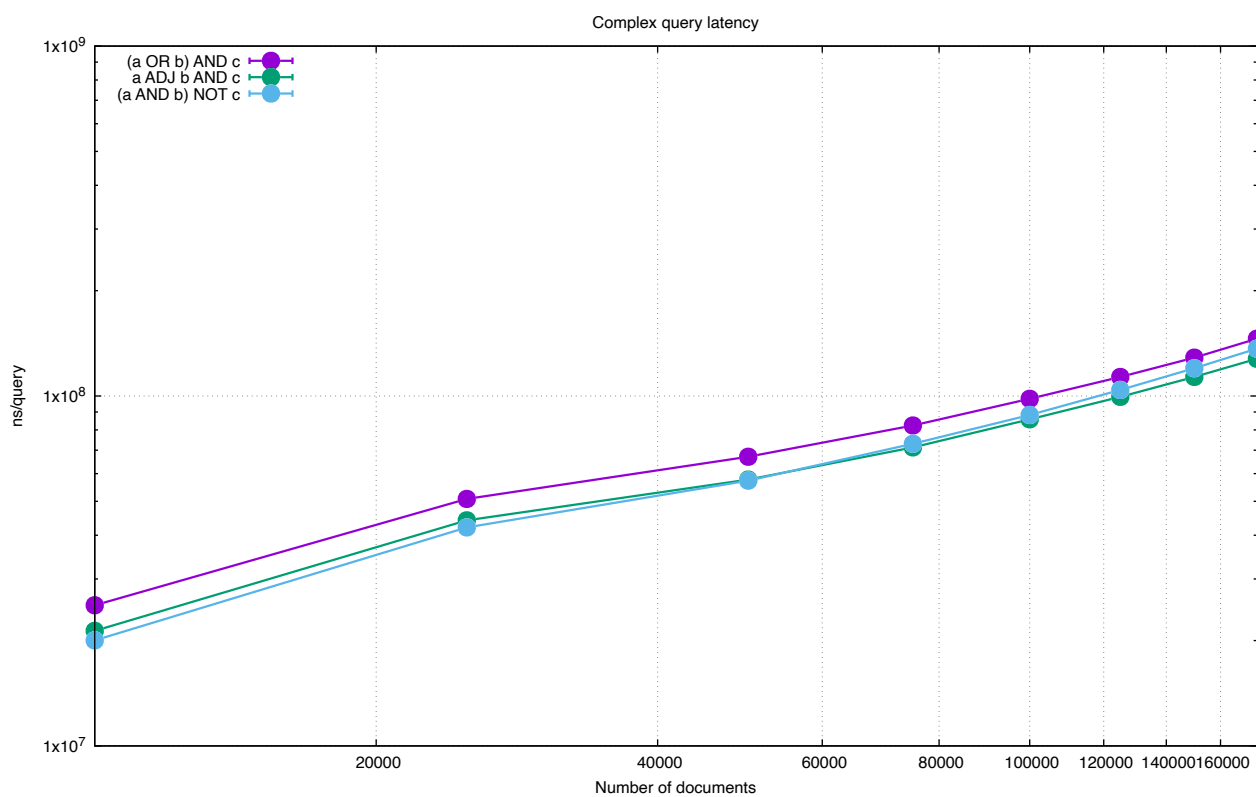


Рисунок 3.4 – Время составных операций поиска

Время поиска растёт с увеличением N для всех операторов, что объ-

ясняется ростом длины posting lists и, соответственно, увеличением объёма вычислений при пересечении/объединении.

Оператор **And** является самым быстрым среди бинарных операторов (для $N = 175000$ его время составляет 57 мс, что в 1.5 раза меньше **Or** (83 мс) и в 2.7 раза меньше **Near** (155 мс)). Операторы **Adj** и **Near** являются самыми медленными (125 мс и 155 мс соответственно при $N = 175000$), поскольку требуют проверки позиций для каждого документа, присутствующего в обоих операндах. Оператор **Near** медленнее **Adj**, так как проверяет позиции в обоих направлениях (в пределах расстояния k) и использует **sortedInsertUnique** для сбора уникальных позиций.

Составные запросы показывают время, сопоставимое с суммой времён базовых операторов, что подтверждает корректность поэтапного вычисления. **Complex_OrAnd** — самый медленный составной запрос (146 мс при $N = 175000$), что объясняется наличием операции **Or** над частотным термом **the** и последующим **And**.

3.6 Размер индекса

График размера индекса в зависимости от числа документов представлен на рисунке 3.5.

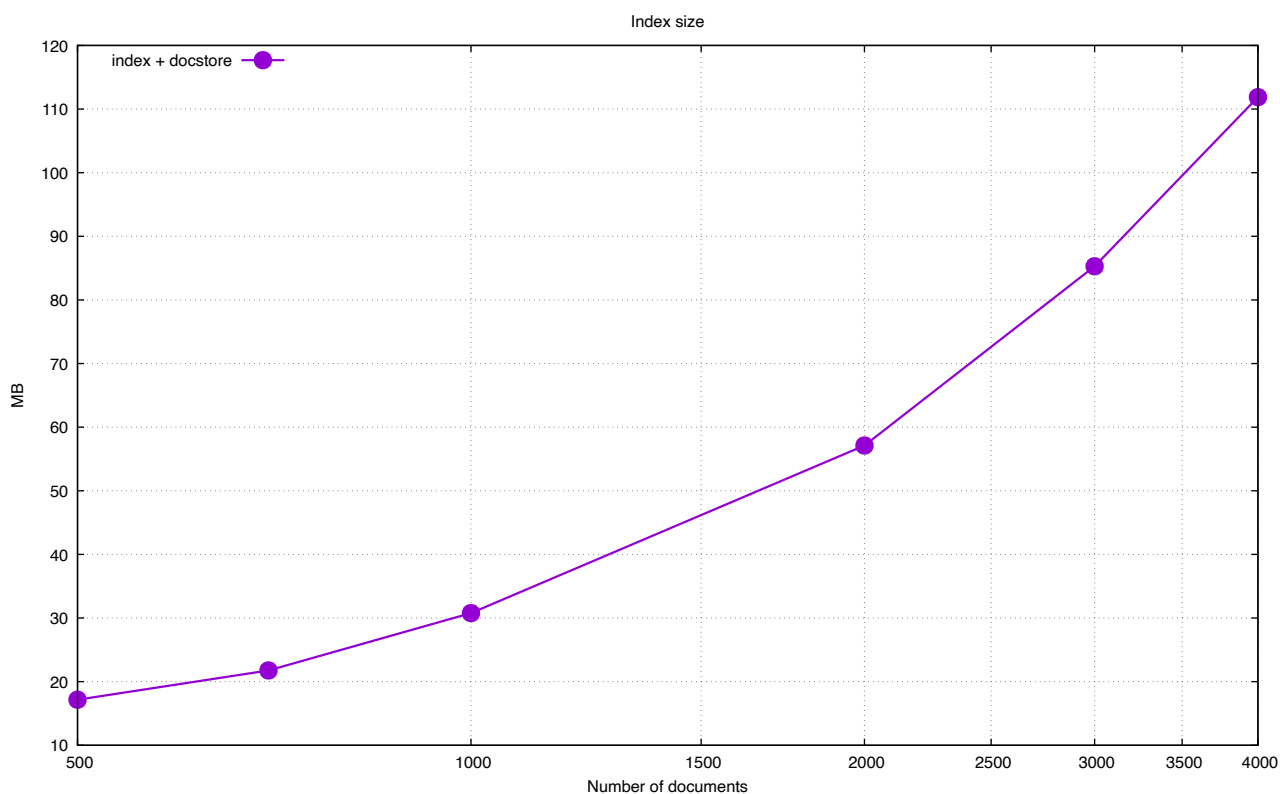


Рисунок 3.5 – Размер индекса в зависимости от числа документов

Размер индекса растёт линейно с числом документов. Это объясняется тем, что основную долю занимают posting lists и данные документов, объём которых пропорционален числу токенов.

3.7 Профилирование

Для профилирования используется отдельная программа `cmd/profile/main.go`, которая выполняет построение индекса или поисковые запросы с включённым `pprof.StartCPUProfile`.

На рисунках 3.6 и 3.7 представлены графы вызовов CPU-профиля для построения индекса и поиска соответственно.

File: profile
Type: cpu
Time: 2026-05-23 23:40:59 MSK
Duration: 303.19ms, Total samples = 100ms (32.98%)
Showing nodes accounting for 100ms, 100% of 100ms total

See <https://git.io/JfYMW> for how to read the graph

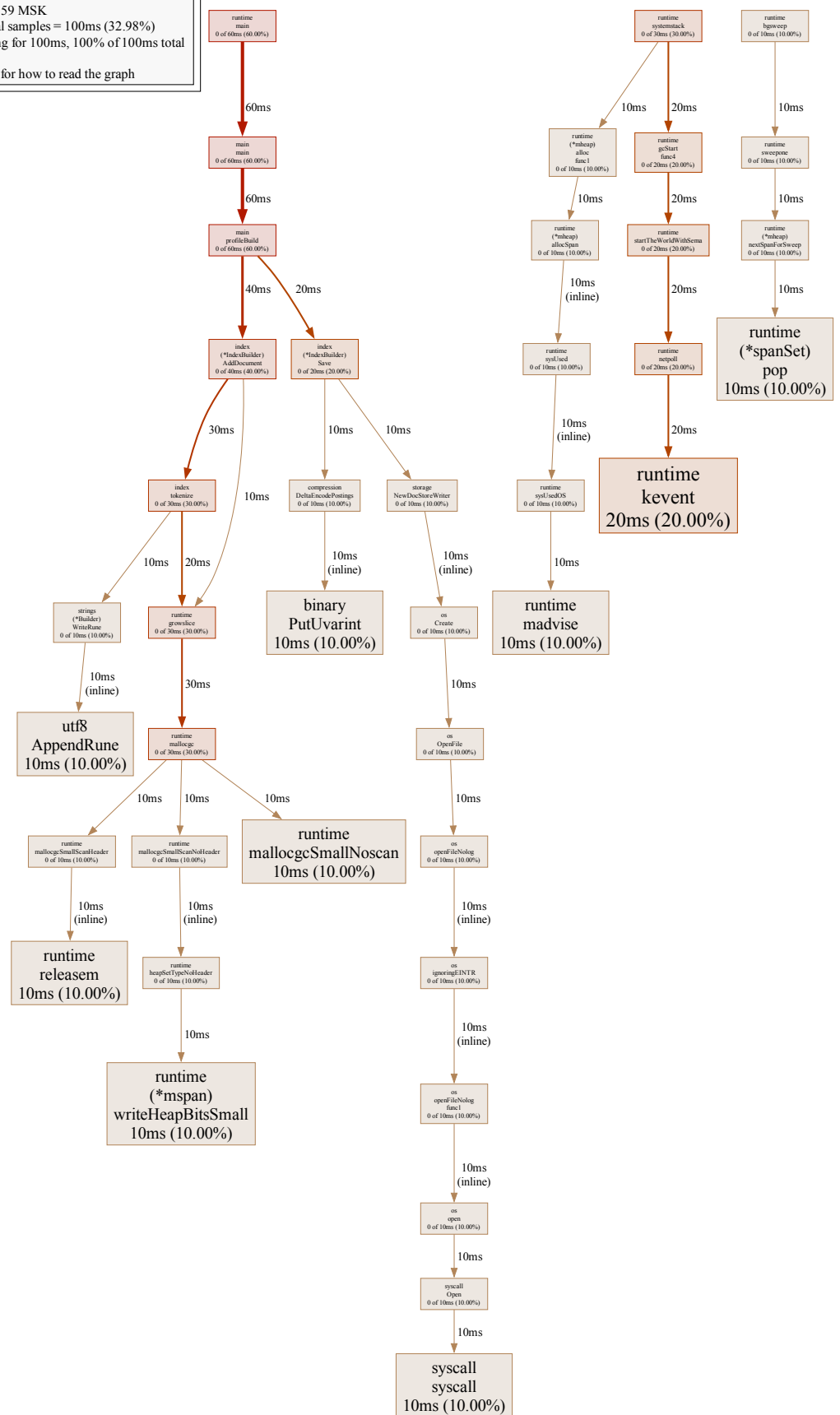


Рисунок 3.6 – CPU-профиль построения индекса

В профиле поиска основное время приходится на:

- `InvertedIndex.GetPostings` — бинарный поиск терма и декодирование posting list;
- `Intersect/Union/Adjacent/Near` — операции над posting lists;
- `Ranker.RankResults` — вычисление BM25-оценок с использованием итераторов.

4 Вывод

Реализован обратный индекс с поддержкой поисковых операторов **AND**, **OR**, **NOT**, **ADJ**, **NEAR/*k***, ранжированием **BM25** и **TUI**-интерфейсом. Индекс хранится на диске в бинарном формате с дельта-кодированием **posting lists** и загружается через **mmap** с ленивым декодированием.

Проведены замеры производительности на наборах данных от 500 до 175000 документов. Можно сделать следующие выводы:

- время построения индекса растёт линейно;
- время **warmpup** значительно превышает время повторного поиска;
- время поиска растёт с увеличением N ; оператор **And** — самый быстрый; **Near** — самый медленный из-за проверки позиций в обоих направлениях;
- размер индекса растёт линейно; дельта-кодирование обеспечивает сжатие в 3–4 раза по сравнению с несжатым хранением.